KEMENTERIAN RISET, TEKNOLOGI, DAN PENDIDIKAN TINGGI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" YOGYAKARTA
JL. SWK 104 (Lingkar Utara) Condongcatur, Yogyakarta 55283 Telp.(0274)486188, 486733, Fax. 486400
Jl. Babarsari 2, Tambakbayan, Yogyakarta 55281 Telp.(0274)486911
Email : info@upnyk.ac.id. Homepage : http://www.upnyk.ac.id

KUIS MATA KULIAH PRAKTIKUM SEMESTER GASAL TA. 2026/2027
PRAKTIKUM IMPLEMENTASI STRUKTUR DATA
PERTEMUAN 06 - Queue (Antrean) dengan Linked List
KERJAKAN SESUAI NAMA FUNCTION PADA STUDENT.CPP

STUDY CASE

Lab komputer kampus punya SATU printer yang dipakai bersama oleh puluhan orang. Anda diminta membuat bagian dalam perangkat lunak pengelola pencetakannya, yang namanya "Cetak Bersama".

Tampilannya sudah jadi. Orang sudah bisa mengirim dokumen, dan papan di layar sudah bisa menampilkan daftarnya. Yang belum ada justru bagian intinya: siapa yang dicetak lebih dulu, dan bagaimana dokumen keluar dari daftar.

Fitur Pertama — Antrean Cetak

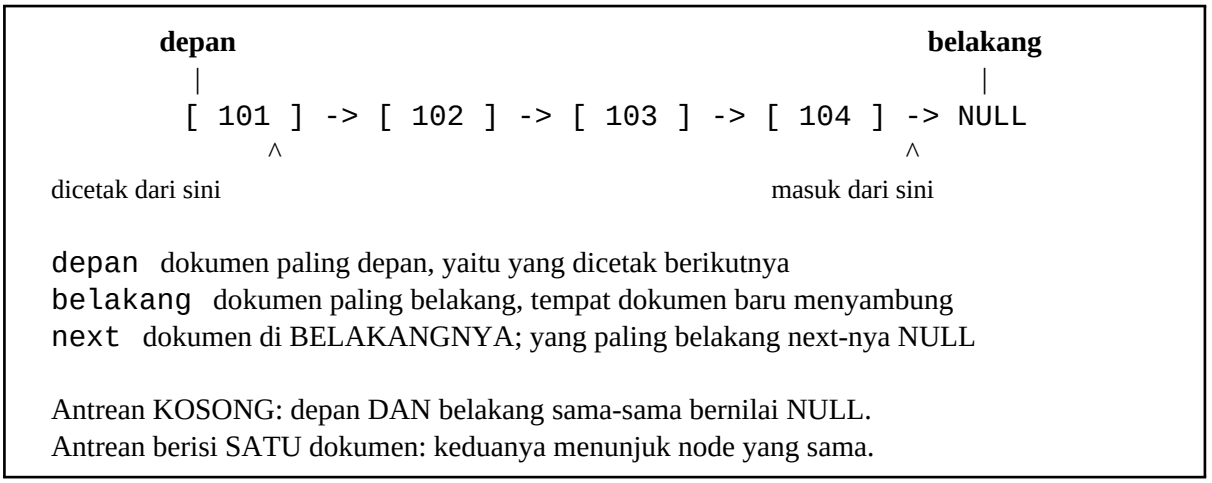
Printernya hanya satu dan hanya bisa mencetak satu dokumen pada satu waktu. Padahal dokumen bisa masuk dari banyak orang sekaligus. Jadi harus ada aturan: siapa yang dicetak lebih dulu?

Coba bayangkan kalau printernya mengambil dokumen yang PALING BARU masuk. Anda mengirim dokumen pertama pagi-pagi, lalu menunggu. Setiap kali printer selesai mencetak, ternyata sudah ada dokumen baru dari orang lain yang lebih muda, dan dokumen itulah yang diambil. Selama masih ada orang yang mengirim, dokumen Anda tidak akan pernah tercetak sampai lab tutup.

Karena itu aturannya harus adil: DOKUMEN YANG PALING PERTAMA MASUK ADALAH DOKUMEN YANG PALING PERTAMA KELUAR. Aturan itu disebut FIFO (First In, First Out). Bentuk penyimpanan yang mengikutinya disebut QUEUE, atau antrean, persis seperti orang mengantre di kasir: yang baru datang berdiri paling belakang, dan yang dilayani selalu yang paling depan.

PERHATIKAN BEDANYA DENGAN PERTEMUAN LALU. Tumpukan piring pada Pertemuan 5 masuk dan keluar dari ujung yang SAMA, yaitu atas. Antrean memakai DUA ujung yang berbeda: dokumen masuk dari belakang, dan keluar dari depan.

Antrean KOSONG ditandai *`depan`* DAN *`belakang`* sama-sama bernilai NULL. Antrean berisi TEPAT SATU dokumen membuat keduanya menunjuk node yang sama:



Kenapa ada dua penunjuk, padahal stack cukup satu? Karena kedua ujungnya dipakai. Kalau yang dipegang hanya *`depan`*, setiap kali ada dokumen baru masuk Anda harus menelusuri dari depan sampai ketemu ujung belakangnya dulu, dan makin panjang antreannya, makin lama. Dengan *`belakang`* yang selalu menunjuk ujung belakang, dokumen baru bisa langsung disambungkan tanpa menelusuri apa pun. Harga dari kemudahan itu: SETIAP OPERASI HARUS MENJAGA KEDUA PENUNJUK TETAP SEPAKAT.

Antrean ini dibangun dari linked list, jadi tidak ada batas banyaknya dokumen dan tidak ada keadaan "penuh". Sebanyak apa pun dokumen yang dikirim, seluruhnya harus tertampung. Yang tetap ada adalah keadaan KOSONG, dan itu sah, bukan kesalahan: printer yang disuruh mencetak padahal tidak ada yang mengantre memang seharusnya tidak melakukan apa-apa.

Fitur Kedua — Tombol "Batalkan Semua"

Lab tutup pukul lima, atau kertasnya habis. Operator menekan tombol "Batalkan Semua", dan seluruh dokumen yang masih mengantre dibuang sekaligus tanpa dicetak.

Yang penting di sini bukan tampilannya. Papan memang akan langsung terlihat bersih begitu kedua penunjuk dikosongkan, tetapi seluruh node dokumennya masih menumpuk di memori, dan tidak ada satu pun yang bisa mencapainya lagi. Keadaan seperti itu disebut KEBOCORAN MEMORI, dan itu ikut dinilai.

FUNCTION YANG SUDAH DISEDIAKAN & TIDAK DINILAI		
`inisialisasi`	Menyiapkan antrean baru menjadi kosong	**Inisialisasi**
`isEmpty`	Apakah antrean sedang kosong	**Cek Kosong**
`front`	Melihat dokumen terdepan tanpa mengeluarkannya	**Peek**
`display`	Membaca seluruh isi antrean menjadi satu baris teks	**Traversal**

Berikut satu sesi pemakaian Cetak Bersama. TIGA langkah bertanda Soal adalah pekerjaan yang harus Anda kerjakan.

1. Empat orang mengirim dokumen berturut-turut. Setiap dokumen berdiri di ujung PALING BELAKANG antrean. **(SOAL 1)**

2. Papan di layar menampilkan dokumen yang akan dicetak berikutnya, dan di bawahnya seluruh isi antrean dari depan ke belakang. (Fungsi untuk ini sudah disediakan, namanya *front* dan *display*.)

3. Printer mencetak. Dokumen PALING DEPAN keluar dari antrean, dan operator perlu tahu nomor dokumen mana yang barusan keluar. Menekan "Cetak" saat tidak ada yang mengantre tidak boleh membuat aplikasi berhenti tidak wajar. **(SOAL 2)**

4. Lab tutup. Tombol "Batalkan Semua" ditekan, dan seluruh dokumen yang masih mengantre dibuang sekaligus. **(SOAL 3)**

1. enqueue (tambah di belakang) — 35 Poin

<table>
<tr><td>Quest</td><td colspan="2">Letakkan sebuah nomor dokumen baru di ujung PALING BELAKANG antrean. Seluruh dokumen yang sudah mengantre tetap tersimpan dengan urutan yang sama persis, hanya saja sekarang mereka semua berada di depan dokumen baru itu.

`<i>next</i>` milik dokumen baru bernilai NULL, dan dokumen yang tadinya paling belakang harus disambungkan ke dokumen baru lewat `<i>next</i>`-nya. Sesudah pemanggilan, `<i>belakang</i>` menunjuk node yang baru. Ada DUA keadaan awal yang diuji: antrean KOSONG membuat `<i>depan</i>` dan `<i>belakang</i>` dua-duanya menunjuk node baru, sedangkan antrean BERISI membuat `<i>depan</i>` TIDAK BOLEH berpindah sedikit pun. Tidak pernah ada penolakan karena penuh, jadi kembaliannya selalu `<i>true</i>` selama node barunya berhasil dibuat.</td></tr>
<tr><td rowspan="3">Parameter/ Variabel</td><td><code>q</code></td><td>Antrean milik pemanggil. Bertanda `&` supaya kedua penunjuknya bisa diubah dari dalam fungsi</td></tr>
<tr><td><code>nomor</code></td><td>Nomor dokumen yang mau dimasukkan</td></tr>
<tr><td><code>return</code></td><td>`<i>true</i>` bila dokumen baru berhasil masuk</td></tr>
<tr><td>Gambaran</td><td colspan="2"><pre>Sebelum : depan -> 101 -> 102 -> NULL   (belakang -> 102)
Operasi : enqueue(q, 103)
Sesudah : depan -> 101 -> 102 -> 103 -> NULL (belakang -> 103)

Sebelum : (kosong)
Operasi : enqueue(q, 101)
Sesudah : depan -> 101 -> NULL
          depan DAN belakang dua-duanya menunjuk node 101</pre></td></tr>
</table>

2. dequeue (keluarkan dari depan) **40 Poin**

<table>
<tr><td>Quest</td><td colspan="2">Keluarkan dokumen PALING DEPAN dari antrean, beri tahu pemanggil nomornya, lalu buang node-nya dari memori. Kembalian menyatakan berhasil atau tidaknya, sedangkan nomor yang keluar disampaikan lewat parameter `<i>nomor</i>` yang bertanda `&`.

Bandingkan dengan front() yang sudah disediakan: bedanya cuma satu, front hanya melihat, sedangkan dequeue benar-benar mengeluarkan dan membuang node-nya.

Isi `<i>nomor</i>` SEBELUM node di-`<i>delete</i>`. INI KESALAHAN PALING SERING: bila yang keluar adalah dokumen TERAKHIR, `<i>belakang</i>` juga harus dijadikan NULL, atau ia akan menunjuk node yang sudah dilepas. Bila antrean kosong (underflow), kembaliannya `<i>false</i>` dan `<i>nomor</i>` TIDAK BOLEH DISENTUH SAMA SEKALI. Ini ikut diuji.</td></tr>
<tr><td rowspan="3">Parameter/ Variabel</td><td>q</td><td>Antrean milik pemanggil. Bertanda `&`</td></tr>
<tr><td>nomor</td><td>Tempat penerima nomor dokumen yang keluar. Bertanda `&`</td></tr>
<tr><td>return</td><td>`<i>true</i>` bila ada dokumen yang dikeluarkan, `<i>false</i>` bila antrean kosong</td></tr>
<tr><td>Gambaran</td><td colspan="2">Sebelum : depan -> 101 -> 102 -> 103 -> NULL
Operasi : int n; dequeue(q, n);
Sesudah : depan -> 102 -> 103 -> NULL, n bernilai 101

Sebelum : depan -> 101 -> NULL (hanya satu dokumen)
Operasi : int n; dequeue(q, n);
Sesudah : (kosong), n bernilai 101
depan DAN belakang dua-duanya harus menjadi NULL

Sebelum : (kosong)
Operasi : int n = -999; dequeue(q, n);
Sesudah : (kosong), dan n TETAP bernilai -999</td></tr>
</table>

3. clear (batalkan semua) **25 Poin**

Quest	Buang seluruh dokumen yang masih mengantre sekaligus, sampai antreannya benar-benar kosong. SELURUH node harus dibuang dengan *`delete`* (banyaknya dihitung checker secara TEPAT), dan alamat dokumen berikutnya harus disimpan SEBELUM node sekarang di-*`delete`*. Menulis *`delete q.depan;`* lalu *`q.depan = q.depan->next;`* berarti membaca memori yang sudah dilepas. Memanggilnya pada antrean yang sudah kosong, atau dua kali berturut-turut, harus aman. Sesudah dikosongkan, antrean harus tetap bisa DIPAKAI LAGI seperti biasa.	
Parameter/ Variabel	`q`	Antrean milik pemanggil. Bertanda `&`
	`return`	Tidak ada (fungsi bertipe *`void`*)

Gambaran	`Sebelum : depan -> 101 -> 102 -> 103 -> NULL`
	`Operasi : clear(q)`
	`Sesudah : (kosong), tiga node dibebaskan`
	`depan dan belakang sama-sama bernilai NULL`	

CATATAN & ATURAN

- Nomor dokumen bertipe *`int`*. Boleh negatif, boleh nol, boleh berulang.
- Antrean yang KOSONG adalah keadaan yang sah, bukan kesalahan.
- Tidak ada kapasitas dan tidak ada keadaan "penuh".
- Node dibuat dengan *`new`*, dan yang keluar dilepas dengan *`delete`*.
- *`depan`* dan *`belakang`* harus selalu sepakat. Tidak boleh ada keadaan di mana yang satu NULL sedangkan yang lain tidak.
- Tidak ada satu pun fungsi yang mencetak ke layar.
- Ketiga soal dinilai SENDIRI-SENDIRI: checker menyiapkan antrean ujinya dengan merangkai node sendiri, tanpa memakai fungsi Anda. Jadi Soal 1 yang belum benar tidak ikut menjatuhkan Soal 2 dan Soal 3. Kecualinya hanya dua test rangkaian FIFO pada Soal 2, yang memang menguji operasi campuran sehingga harus memakai *`enqueue`* Anda.
- *`cin`* hanya boleh dipakai DI DALAM *`main()`*. Jangan menaruh *`cin`* di dalam ketiga fungsi yang dinilai, karena checker memanggil fungsi-fungsi itu tanpa memberi masukan apa pun.